

ARNIFI

Shared UAE Phone Bell

Advanced Product + Technical Requirements Document

Targetter website -> Firebase API -> FCM -> Target Samsung APK -> acknowledgement

```
TARGETTER WEBSITE
  |
  v
HTTPS API / CLOUD FUNCTION
  |
  v
FIREBASE CLOUD MESSAGING (FCM)
  |
  v
TARGET SAMSUNG ANDROID APK
  |
  v
RING + VIBRATION + REQUESTER NAME
  |
  v
ACKNOWLEDGEMENT -> FIRESTORE -> WEBSITE
```

Version: 2.1 - consolidated implementation PRD

Date: September 2026

Primary users: ~50 Arnifi employees / Targetters

Target device: One shared Samsung Android phone

Backend: Firebase Cloud Functions v2 + Firestore + FCM

Android: Native Kotlin APK

Website: Wordle / chosen web builder with JavaScript + API support

Source of truth: This document supersedes the earlier draft PRDs in this conversation and consolidates the Target / Targetter model, API/webhook-style trigger, FCM delivery, Firestore state synchronization, Android APK behavior, implementation phases, and Claude Code/Codex prompts.

1. Executive Summary

Arnifi has one shared Samsung Android phone used for UAE-related calling. Roughly 50 employees may need the phone, but today a requester has to physically ask around the office to locate it. This product replaces that manual process with a one-click request flow.

A Targetter opens a company URL, identifies themselves once, and presses RING PHONE. A secure Firebase backend creates the request and sends a high-priority Firebase Cloud Messaging event to the one Target Samsung device. The Android APK receives the event, plays a distinctive ringtone, vibrates, and shows the requester name. The person holding the Samsung can press I'VE GOT IT; that acknowledgement is written back to Firebase and appears in realtime on the Targetter website.

Core architectural rule: FCM is the trigger that reaches the Samsung. Firestore is the source of truth for request state and realtime UI synchronization. The Samsung must not continuously poll Firestore as the primary ringing mechanism.

1.1 Product Goal

Reduce the process of locating the shared UAE calling phone from an office-wide search to a fast digital request and acknowledgement flow:

Open URL -> identify user -> RING PHONE -> Samsung alerts -> holder acknowledges -> requester sees confirmation

1.2 Success Criteria

- **MVP outcome** - one website action reliably causes one Samsung to ring and show the requester name.
- **Friction** - Targetters do not need to install an app; only the shared Samsung requires an APK.
- **Clarity** - the website distinguishes Sent, Delivered, and Acknowledged instead of assuming an FCM send means handset delivery.
- **Safety** - one request should cause one ring; duplicate clicks, duplicate FCM messages, and spam should not create repeated alerts.
- **Operability** - the Samsung app provides diagnostics and a Test Ring function so the team can isolate Firebase, network, notification, and audio problems.

1.3 Scope at a Glance

Surface / Service	Purpose	MVP
Targetter web page	Name setup, Ring Phone button, request status, acknowledgement	P0
Firebase HTTPS API / Cloud Functions	Authentication, validation, rate limits, state transitions, FCM send	P0
Cloud Firestore	Users, devices, requests, configuration, realtime state	P0
Firebase Cloud Messaging	Push trigger from backend to Samsung target	P0
Samsung Android APK	Receive FCM, ring, vibrate, show requester, acknowledge	P0
Diagnostics / telemetry	Test Ring, app health, delivery events, logs	P0/P1
Google Workspace login, TTS, admin analytics	Security and operational enhancements	P1/P2

2. Product Context, Users, and Terminology

2.1 Problem Statement

The company has one shared Indian-SIM Samsung phone used for UAE-related calls. Because the device is physically mobile and can be carried by any employee, people regularly need to ask multiple colleagues who currently has it. The problem is not making the UAE call itself; the problem is quickly locating the shared device when someone needs it.

2.2 Target

The Target is the physical shared device being requested. For Version 1 there is exactly one Target:

```
deviceId: uae-phone-01
displayName: UAE Calling Phone
platform: Android
role: target
```

A future version may support additional Target devices such as a KSA phone, Qatar phone, reception phone, or devices in other offices. The Version 1 schema should not block that expansion.

2.3 Targetter

A Targetter is an employee who wants the Target phone. Approximately 50 employees may act as Targetters. They access the system through the website and do not need a native application.

2.4 Why Devices and Users Are Separate

The Samsung should not be stored as a fake employee/user. People and devices are different entities with different security and lifecycle needs. Users have identities and profiles; devices have FCM tokens, app versions, health metadata, and capabilities.

Entity	Collection	Examples
Targetter / person	users/{uid}	Rahul, Ayush, Raj
Target / device	devices/{deviceId}	uae-phone-01
Request	phoneRequests/{requestId}	Rahul requests uae-phone-01

2.5 "Webhook" Terminology

The team may casually refer to the browser-to-backend trigger as a webhook. Technically, this product should use a normal authenticated HTTPS API implemented with Firebase Cloud Functions v2. No third-party automation service is required.

Targetter Website -> HTTPS API / Cloud Function -> FCM -> Target Samsung

- **Do use** - an HTTPS endpoint or Firebase callable function as the backend trigger.
- **Do not use** - Zapier, Make, n8n, or similar systems unless there is a separate integration requirement.
- **Do not expose** - Firebase Admin credentials or server-side FCM sending capability to the browser.

3. Product Principles and Non-Negotiable Architecture

3.1 Separation of Responsibilities

Layer	Responsibility
Website	Capture user intent and display request state
API / Cloud Functions	Authenticate, validate, rate-limit, create state, send FCM, enforce transitions

Layer	Responsibility
Firestore	Source of truth for users, devices, request state, configuration, events
FCM	Time-sensitive trigger from backend to target Android device
Android APK	Physical response: notification, ring, vibration, optional TTS, acknowledgement

3.2 FCM Is the Trigger; Firestore Is the State

WRONG PRIMARY FLOW

Samsung continuously checks Firestore -> sees new document -> rings

REQUIRED PRIMARY FLOW

Website -> Cloud Function -> FCM -> FirebaseMessagingService -> ring

REALTIME STATE FLOW

Android/API -> Firestore -> website realtime listener

Firestore realtime listeners remain useful for the Targetter website because they provide a clean mechanism for status updates. They should not be treated as the mechanism that wakes a sleeping Android phone and starts the alert.

3.3 One App vs One System

The entire solution is one product and one Firebase-backed system, but it has two user-facing surfaces. The 50 employees use a web page. Only the Target Samsung has a native Android APK. This minimizes deployment friction while preserving Android-specific background and notification behavior where it is needed.

3.4 First Milestone

Milestone 1: Do not begin with a polished website or a complete app. The first proof must be: HTTP request -> Cloud Function -> FCM -> Android FirebaseMessagingService -> Samsung receives requester name.

4. End-to-End User Experience

4.1 First-Time Targetter Flow

1. Employee opens the internal phone request URL, for example phone.arnifi.com.
2. If no saved profile exists, the page asks for the employee name.
3. The name is validated, stored in localStorage, and associated with the current Firebase user UID.
4. The employee lands on the main request screen.

UAE Calling Phone

What is your name?

[Rahul Sharma]

[Continue]

4.2 Returning Targetter Flow

The browser remembers the employee name so the page can go directly to the request action. A Change name option must remain available.

Hi Rahul

Need the shared UAE calling phone?

[RING PHONE]

Change name

4.3 Ring Request Flow

5. Targetter presses RING PHONE.
6. The browser immediately disables the button and creates a clientRequestId UUID for idempotency.
7. The browser calls the Firebase HTTPS API.
8. Backend authenticates the user, validates the request, enforces cooldown/concurrency, creates the Firestore request, and sends an FCM event.
9. The website shows Request sent only after the backend confirms the FCM send attempt was accepted.
10. When the Android app receives the FCM event, it sends a Delivered callback and begins the physical alert.
11. The website realtime listener changes the status to Phone received your request.
12. The person holding the Samsung presses I'VE GOT IT.
13. Backend validates the acknowledgement, writes the Acknowledged state, and the Targetter website updates in realtime.

4.4 Target Device Experience

Incoming request

UAE Phone Requested

Rahul Sharma needs the UAE calling phone.

[I'VE GOT IT]

Behavior: custom sound + vibration + visible notification

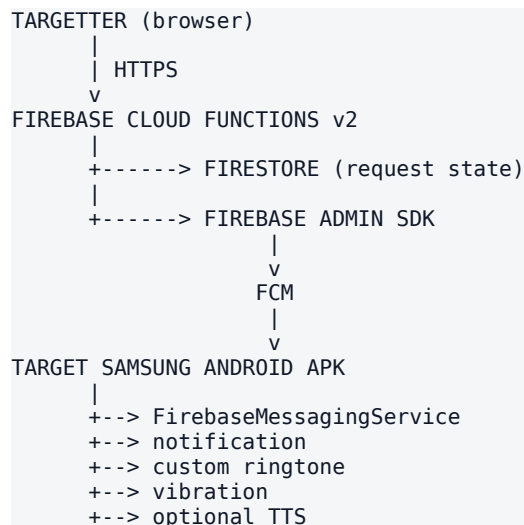
Optional P1: text-to-speech requester name

4.5 Requester Status UX

Backend / Device State	Targetter UI
Creating	Sending request...
Sent	Request sent
Delivered	Phone received your request
Acknowledged	Someone has acknowledged your request
Cancelled	Request cancelled
Expired	No one acknowledged. Ring again?
Failed	Unable to send the request. Please try again.

5. System Architecture

5.1 High-Level Architecture





5.2 Firebase Project

Use one Firebase project per environment. Recommended environments:

```

arnifi-phone-bell-dev
arnifi-phone-bell-prod
  
```

Firestore Service	Use
Firebase Authentication	Targetter identity; anonymous auth for MVP, Workspace later
Cloud Firestore	Users, devices, phoneRequests, config, requestEvents
Cloud Functions v2	HTTPS API and trusted business logic
Firebase Cloud Messaging	Push trigger to target Samsung
Firebase Hosting	Optional if Wordle / chosen builder is not used for final hosting
Crashlytics	Android crash and reliability monitoring
App Check	P1 protection against unauthorized clients where practical

5.3 Website Technology

The requester has selected Wordle / a web-builder workflow for the website. The only hard technical requirement is that the page can run JavaScript, perform authenticated HTTPS calls, initialize Firebase web SDK functionality as needed, and subscribe to Firestore request state. If the builder cannot support that reliably, host this internal utility page separately on Firebase Hosting or another suitable web host and map the company subdomain to it.

5.4 Android Technology

Area	Recommendation
Language	Kotlin
UI	Jetpack Compose
Architecture	MVVM / clear repository + service separation
Push	Firebase Messaging / FirebaseMessagingService
Audio	Dedicated RingController / foreground audio service when required
Local settings	DataStore
Background health	WorkManager where appropriate, not as the FCM trigger
Monitoring	Firebase Crashlytics

6. Web Application Requirements

6.1 URL and Access

Recommended internal URL: phone.arnifi.com or ring.arnifi.com. The interface must work on desktop and mobile browsers because employees may request the phone from either.

6.2 First-Visit Profile

- Ask for display name only when no local profile is present.
- Minimum 2 characters; maximum 50 characters.

- Trim leading/trailing whitespace.
- Reject or escape HTML/script content.
- Persist the display name in localStorage.
- Also write/update users/{uid} after authentication.
- Provide a Change name action.

localStorage key: `arnifi_phone_display_name`

6.3 Authentication

MVP may use Firebase Anonymous Authentication so each browser gets a UID without adding login friction. The profile name should be associated with that UID. P1 should evaluate Google Workspace authentication and domain restriction for stronger identity assurance.

6.4 Main Screen

- Show the current Target name: UAE Calling Phone.
- Show the remembered Targetter name.
- Primary CTA: RING PHONE.
- Disable the CTA while a request is active or while the API call is in flight.
- Display status changes using Firestore realtime subscription.
- Offer Cancel Request when allowed.
- Show a warning if device health suggests the Target may be offline, but do not block Ring based only on heartbeat staleness.

6.5 Idempotency on the Web

Before calling the create request endpoint, the browser generates a UUID `clientRequestId`. If a request times out client-side and the browser retries the same action, reuse that UUID until the original outcome is known. This prevents double-clicks and network retries from creating duplicate FCM sends.

6.6 Realtime Subscription

After the backend returns `requestId`, subscribe to `phoneRequests/{requestId}`. The UI must react to: sent, delivered, acknowledged, cancelled, expired, and failed. Do not require manual refresh.

6.7 Error UX

Condition	User-facing message
No internet	You appear to be offline. Connect to the internet and try again.
API error	Unable to send the phone request. Please try again.
Active request exists	The UAE phone is already being requested. Please wait for the current request to finish.
Delivery not confirmed	Request sent. Waiting for the phone to receive it...
Target possibly offline	The UAE phone may be offline. We will still attempt delivery.
Expired	No one acknowledged your request. Ring again?

7. Backend / API Requirements

7.1 API Principle

The browser never sends FCM directly. All FCM sends, device-token access, request state transitions, rate limits, and privileged validation occur in Cloud Functions using the Firebase Admin SDK.

7.2 Required Endpoints

Method	Endpoint	Purpose
POST	/api/v1/phone-requests	Create request and send FCM
POST	/api/v1/phone-requests/:id/delivered	Target confirms FCM receipt
POST	/api/v1/phone-requests/:id/acknowledge	Target holder acknowledges request
POST	/api/v1/phone-requests/:id/cancel	Targetter cancels active request
POST	/api/v1/devices/register	Target registers / refreshes FCM token
POST	/api/v1/devices/heartbeat	Target updates health timestamp

7.3 Create Phone Request

```

POST /api/v1/phone-requests

Request:
{
  "clientRequestId": "78a08308-5c43-4b80-b155-12d28c652fee",
  "targetDeviceId": "uae-phone-01"
}

Response:
{
  "success": true,
  "requestId": "req_abc123",
  "status": "sent"
}

```

Server flow: authenticate -> validate -> App Check where enabled -> idempotency check -> cooldown check -> active request check -> resolve current user profile -> load target device -> create request -> send FCM -> update sentAt/status -> return requestId.

7.4 Delivered Callback

```

POST /api/v1/phone-requests/req_abc123/delivered
{
  "deviceId": "uae-phone-01"
}

```

Only the authorized Target device may transition a matching active request from sent to delivered. The backend writes deliveredAt using a server timestamp.

7.5 Acknowledge Callback

```

POST /api/v1/phone-requests/req_abc123/acknowledge
{
  "deviceId": "uae-phone-01"
}

```

The backend validates target identity, request state, targetDeviceId, and expiry. A duplicate acknowledgement should be idempotent and must not create an error visible to the holder.

7.6 Cancel Request

A Targetter may cancel their own active request. The backend transitions the request to cancelled and optionally sends an FCM cancellation event to the Samsung so any currently active ring can stop immediately.

7.7 Device Registration

```

POST /api/v1/devices/register
{
  "deviceId": "uae-phone-01",
  "fcmToken": "...",
  "appVersion": "1.0.0",

```

```
}
  "deviceModel": "Samsung"
}
```

The target APK calls this at initial setup and whenever Firebase invokes `onNewToken()`. Never hardcode the FCM token.

8. Firestore Data Model

8.1 Collections

```
users/
devices/
phoneRequests/
config/
requestEvents/ (recommended for observability)
```

8.2 users/{uid}

```
{
  "displayName": "Rahul Sharma",
  "role": "targetter",
  "createdAt": "SERVER_TIMESTAMP",
  "updatedAt": "SERVER_TIMESTAMP",
  "lastSeenAt": "SERVER_TIMESTAMP"
}
```

8.3 devices/{deviceId}

```
{
  "deviceId": "uae-phone-01",
  "displayName": "UAE Calling Phone",
  "deviceType": "target",
  "platform": "android",
  "active": true,
  "fcmToken": "...",
  "appVersion": "1.0.0",
  "deviceModel": "Samsung",
  "lastSeenAt": "SERVER_TIMESTAMP",
  "lastTokenRefreshAt": "SERVER_TIMESTAMP",
  "capabilities": {
    "ring": true,
    "vibration": true,
    "tts": true
  }
}
```

Security: Normal Targetters must never be able to read the FCM token or privileged device metadata.

8.4 phoneRequests/{requestId}

```
{
  "requestId": "req_abc123",
  "clientRequestId": "uuid",
  "targetDeviceId": "uae-phone-01",
  "requestedBy": {
    "uid": "uid_123",
    "name": "Rahul Sharma"
  },
  "status": "sent",
  "createdAt": "SERVER_TIMESTAMP",
  "sentAt": "SERVER_TIMESTAMP",
  "deliveredAt": null,
  "acknowledgedAt": null,
  "cancelledAt": null,
  "expiresAt": "TIMESTAMP",
  "failureCode": null,
  "failureMessage": null
}
```

8.5 config/sharedPhone

```
{
  "targetDeviceId": "uae-phone-01",
  "ringDurationSeconds": 10,
  "requestTimeoutSeconds": 60,
  "targetterCooldownSeconds": 20,
  "allowConcurrentRequests": false,
  "vibrationEnabled": true,
  "ttsEnabled": true,
  "deliveryTimeoutSeconds": 15
}
```

8.6 requestEvents/{eventId}

Recommended event log values: REQUEST_CREATED, FCM_SEND_STARTED, FCM_SEND_SUCCEEDED, FCM_SEND_FAILED, FCM_RECEIVED, RING_STARTED, RING_STOPPED, REQUEST_ACKNOWLEDGED, REQUEST_CANCELLED, REQUEST_EXPIRED.

```
{
  "requestId": "req_abc123",
  "event": "FCM_RECEIVED",
  "createdAt": "SERVER_TIMESTAMP",
  "deviceId": "uae-phone-01"
}
```

9. Request State Machine

9.1 Canonical States

```

CREATED
  |
  v
SENT
  |
  v
DELIVERED
  |
  v
ACKNOWLEDGED

```

Alternate terminal paths:
FAILED / CANCELLED / EXPIRED

9.2 State Meaning

State	Meaning
created	Firestore request exists; FCM has not yet been successfully submitted
sent	Firebase accepted the FCM send request; handset receipt is not yet proven
delivered	Android APK actually received the matching FCM message and confirmed it
acknowledged	A person holding the Samsung pressed I'VE GOT IT
cancelled	Requester cancelled before completion
expired	No acknowledgement was received before expiresAt
failed	Backend or delivery preparation failed in a way that prevents normal continuation

9.3 Allowed Transitions

```

created -> sent
created -> failed

sent -> delivered
sent -> cancelled
sent -> expired
sent -> failed

delivered -> acknowledged
delivered -> cancelled
delivered -> expired

```

Terminal states are not moved back to earlier states.

State transitions must be enforced server-side. Clients should never be permitted to arbitrarily set status, deliveredAt, or acknowledgedAt.

10. Firebase Cloud Messaging Requirements

10.1 Message Purpose

FCM is used only to trigger time-sensitive user-visible target-device behavior. The message should carry the minimum information the Android app needs to identify the request, display the requester name, and call back to the backend.

10.2 Conceptual Payload

```

{
  "android": {
    "priority": "high"
  },
  "data": {
    "type": "PHONE_REQUEST",
    "requestId": "req_abc123",
    "targetDeviceId": "uae-phone-01",
    "requestedByName": "Rahul Sharma"
  }
}

```

10.3 Cancellation Payload

```

{
  "data": {
    "type": "PHONE_REQUEST_CANCELLED",
    "requestId": "req_abc123"
  }
}

```

10.4 Delivery Semantics

- FCM send success means Firebase accepted the send request; it is not handset delivery confirmation.
- The Android app must call the Delivered API as soon as the request is received and validated.
- The website may show Sent before Delivered, but should not claim the phone definitely rang before device receipt is confirmed.
- All FCM payload fields should be treated as untrusted until validated against expected message type and request identity.

10.5 Token Lifecycle

The Android FirebaseMessagingService must implement onNewToken(). A refreshed token is sent to the register-device endpoint and stored on the target device record. The backend always loads the latest active token at send time.

11. Android APK Requirements

11.1 APK Purpose

The Android APK exists only on the shared Samsung and acts as the physical alert endpoint. It should be small, native, resilient, and easy to diagnose. It does not need to be published to the Play Store for MVP.

11.2 Suggested Package and App Name

App: Arnifi Phone Bell
Package: com.arnifi.phonebell

11.3 Incoming Message Pipeline

```
FCM
|
v
FirebaseMessagingService.onMessageReceived()
|
+--> validate message type
+--> validate requestId / targetDeviceId
+--> deduplicate requestId
+--> call Delivered API
+--> create visible notification
+--> start ring behavior
    |
    +--> ringtone
    +--> vibration
    +--> optional TTS
    +--> stop on timeout / acknowledge / cancel
```

11.4 Android Module Structure

```
com/arnifi/phonebell/
  data/
    api/
    model/
    repository/
  firebase/
    ArnifiFirebaseMessagingService.kt
    DeviceRegistrationManager.kt
  notifications/
    NotificationManager.kt
    NotificationChannels.kt
  ring/
    RingController.kt
    RingService.kt
    VibrationController.kt
    TextToSpeechController.kt
  requests/
    RequestRepository.kt
    RequestDeduplicator.kt
  ui/
    MainActivity.kt
    screens/HomeScreen.kt
    screens/DiagnosticsScreen.kt
  settings/
    AppSettings.kt
  workers/
    HeartbeatWorker.kt
```

11.5 Notification Channel

Channel ID: uae_phone_requests
Display name: UAE Phone Requests
Importance: HIGH

The channel should use the distinctive bundled sound and vibration behavior. The app must account for Android runtime notification permission on versions that require it and surface a clear health warning when notifications are disabled.

11.6 Custom Ring

Resource example:
android/app/src/main/res/raw/phone_request.mp3

- Distinctive and immediately recognizable in the office.
- Professional and short.
- Different from WhatsApp, common phone ringtones, and standard notification sounds.
- Bundled in the APK rather than downloaded at runtime.
- Default ring duration: 10 seconds; remotely configurable within a sensible range.

11.7 Vibration and Optional TTS

Vibration is part of the P0 alert. Text-to-speech is P1 and may say, for example, "Rahul Sharma is requesting the UAE phone." TTS failure must never block the core sound/notification flow.

11.8 Acknowledge Action

The notification should expose an I'VE GOT IT action where feasible. Pressing it stops the current ring/vibration, calls the acknowledge endpoint, and updates/dismisses the notification. Repeated acknowledgement taps must be safe and idempotent.

11.9 Duplicate Protection

The app stores a short history of recently handled request IDs. If the same FCM request is delivered twice, the second event must not trigger another ring. A cancellation event must only stop the alert associated with the matching requestId.

11.10 Android Permissions / Configuration

- INTERNET.
- POST_NOTIFICATIONS where required by target Android version.
- VIBRATE.
- Foreground service declarations / permissions required by the chosen audio implementation and target SDK.
- Background data enabled.
- App battery usage configured to unrestricted where appropriate on the Samsung.
- Keep the app out of sleeping/deep-sleep app lists where the device exposes those controls.

12. Android Diagnostics and Operations

12.1 Main Health Screen

ARNIFI PHONE BELL

Status: READY
Device: UAE Calling Phone

Firebase	[OK]
Notifications	[OK]
Internet	[OK]
Audio	[OK]

Last request: Rahul Sharma - 2:14 PM

[TEST RING]
[DIAGNOSTICS]

12.2 Diagnostic Fields

- Device ID.
- App version.
- Target environment (dev / prod).
- FCM token registered state; never display full token unless explicitly in a developer-only debug build.
- Notification permission status.
- Internet status.
- Last heartbeat time.
- Last FCM received time.
- Last request ID / requester name / status.
- Audio/ring test status.

12.3 Test Ring

TEST RING is a mandatory P0 troubleshooting feature. It must call exactly the same local RingController used by a real FCM request, without needing Firebase.

Observation	Likely problem area
Test Ring works, real request does not	FCM, network, token registration, backend, permissions
Test Ring fails	Android audio / vibration / notification implementation or device settings

12.4 Heartbeat

The Android app may update devices/uae-phone-01.lastSeenAt periodically using a backend endpoint / WorkManager. Heartbeat is diagnostic only and should not be treated as mathematically exact device availability.

lastSeenAt age	Website interpretation
< 5 minutes	Likely online
5-15 minutes	Unknown / potentially delayed heartbeat
> 15 minutes	Possibly offline

13. Reliability, Concurrency, and Abuse Protection

13.1 Idempotency

Every create request includes a clientRequestId UUID. The backend stores a uniqueness record or queries by requester + clientRequestId. Repeating the same idempotency key returns the original request rather than sending another FCM.

13.2 User Cooldown

Recommended MVP cooldown: one new request per Targetter per 20 seconds. The UI should disable repeat actions while a request is active.

13.3 Concurrent Request Policy

MVP should allow only one active request for the shared target at a time. If Rahul has an active request, Ayush should see that the phone is currently being requested and should wait. Do not build a queue before the core behavior is proven.

13.4 Expiration

Recommended request timeout: 60 seconds. A server-side or scheduled mechanism should transition stale active requests to expired. The Targetter can then create a new request.

13.5 Ring Timeout

The physical audio alert must never continue indefinitely. Recommended default: 10 seconds. Stop the sound earlier on acknowledgement or cancellation. The notification may remain visible after the audio stops.

13.6 Delivery Timeout

Recommended UI delivery confirmation window: approximately 15 seconds. If the request remains sent without a Delivered callback, show Waiting for device / Delivery could not yet be confirmed rather than falsely claiming success.

13.7 Retry Strategy

Retries must preserve idempotency. The browser should not generate a new `clientId` merely because the network response was lost. FCM sends should not be automatically repeated without confirming whether the original request may already have reached the device.

14. Security Requirements

14.1 Minimum Controls

- HTTPS only.
- Firebase Authentication for web users.
- Server-side validation of every request and state transition.
- Cloud Functions / Admin SDK as the only FCM sender.
- FCM tokens and privileged device metadata hidden from normal users.
- Firestore security rules that limit users to appropriate reads and prevent direct privileged writes.
- Rate limits and idempotency.
- Secrets stored outside Git.
- App Check where practical as a P1 hardening measure.

14.2 Authentication Roadmap

Phase	Approach
MVP	Firebase Anonymous Authentication + saved display name
P1	Google Workspace sign-in restricted to company domain
Future	Directory-backed employee identity and richer authorization if required

14.3 Device Trust

Device registration must not allow any arbitrary client to replace the production target FCM token. The implementation should use a target-device bootstrap secret, admin provisioning flow, Firebase Auth identity, or another controlled mechanism appropriate to the internal deployment. Because the APK may be sideloaded, do not make Play Store-only trust mechanisms a blocker for Version 1.

14.4 Secrets and Logging

- Never commit Firebase service-account JSON, release keystore passwords, private API secrets, or signing credentials.
- Never log full FCM tokens, auth tokens, or secrets in production logs.
- Cloud Functions logs should include `requestId`, `targetDeviceId`, event, latency, and FCM outcome without sensitive credentials.

15. Error and Edge-Case Handling

Scenario	Required behavior
Browser double-click	One backend request, one FCM, one ring
Network retry	Reuse clientRequestId; return original request if already created
Duplicate FCM	Android deduplicates requestId and does not ring twice
Old/stale FCM token	Record failure; device must refresh token via onNewToken / registration
Samsung offline	Do not claim Delivered; request eventually expires
Notifications disabled	Diagnostics show warning and provide route to settings
Targetter cancels	Request becomes cancelled; matching active ring stops
Acknowledgement after expiry	Reject or safely no-op according to state-machine policy
Two employees request at same time	Backend atomically allows only one active request
Firebase send accepted but device never confirms	Remain Sent / waiting; do not present as delivered
TTS engine fails	Keep ringtone, vibration, notification working
App process restarted	FCM token lifecycle and request handling recover normally

15.1 Device Offline Copy

UX wording: Prefer "The UAE phone may be offline. We will still attempt delivery." Do not block a request merely because lastSeenAt is stale.

16. Observability and Metrics

16.1 Event Logging

Use Cloud Functions structured logs and optionally requestEvents to create a debuggable timeline for each request. Every log/event should contain requestId and a safe event name.

16.2 Product Metrics

Metric	Why it matters
Requests created	Adoption / usage volume
FCM send success rate	Backend-to-Firebase reliability
Delivery confirmation rate	Actual target receipt reliability
Median time to Delivered	Speed from click to device receipt
Acknowledgement rate	Whether the office holder is responding
Median time to Acknowledged	Real operational improvement
Expired requests	Potential device availability / process issues
Failed requests	Technical reliability

16.3 Reliability Target

A reasonable internal objective is for the large majority of normal online requests to reach the Samsung within a few seconds. The UI and requirements must still acknowledge that internet connectivity, device power management, permissions, and push delivery can prevent or delay immediate receipt. The system therefore uses explicit Delivered confirmation instead of assuming success.

17. Repository and Code Organization

17.1 Monorepo

```

arnifi-phone-bell/
|-- README.md
|-- PRD.md
|-- ARCHITECTURE.md
|-- SECURITY.md
|-- package.json

-- web/
|  |-- src/
|  |  |-- api/
|  |  |-- components/
|  |  |-- firebase/
|  |  |-- hooks/
|  |  |-- pages/
|  |  |-- types/
|  |-- package.json
|
|  -- functions/
|  |  |-- src/
|  |  |  |-- api/
|  |  |  |-- firebase/
|  |  |  |-- services/
|  |  |  |-- validation/
|  |  |-- types/
|  |-- package.json
|
|  -- android/
|  |  |-- app/
|  |  |-- gradle/
|  |-- settings.gradle.kts
|
|  -- firebase/
|  |  |-- firestore.rules
|  |  |-- firestore.indexes.json
|  |-- firebase.json
|
|-- docs/
|  |-- android-installation.md
|  |-- firebase-setup.md
|  |-- testing.md
|  |-- release-apk.md
|  |-- troubleshooting.md

```

17.2 Backend Stack

```

Node.js
TypeScript
Firebase Functions v2
Firebase Admin SDK
Zod (recommended validation)

```

17.3 Backend Service Abstraction

```

interface PushNotificationService {
  sendPhoneRequest(
    deviceToken: string,
    request: PhoneRequest
  ): Promise<void>;
}

```

Implementation: `FirebasePushNotificationService`

Keep FCM transport logic separate from HTTP handlers so the API remains testable and future transport changes do not require rewriting request business logic.

17.4 Web API Abstraction

```
PhoneRequestApi
createRequest()
cancelRequest()
subscribeToRequest()
```

Do not scatter raw fetch calls and Firestore listeners through UI components.

18. Priority and Roadmap

18.1 P0 - Must Work for MVP

- Firebase dev/prod structure.
- Firestore schema and rules.
- Target device registration and FCM token refresh.
- createPhoneRequest API.
- High-priority FCM send.
- Android FirebaseMessagingService.
- Custom ringtone and vibration.
- Requester name in notification.
- Delivered callback.
- I'VE GOT IT acknowledgement.
- Realtime Targetter status.
- First-time name setup and localStorage persistence.
- Idempotency and cooldown.
- Single active request policy.
- Request expiration.
- Test Ring diagnostics.
- Release-signed APK and installation procedure.

18.2 P1 - Important Enhancements

- Text-to-speech requester name.
- Cancel request FCM.
- Device heartbeat and online warning.
- Crashlytics and richer diagnostics.
- Request event history / admin diagnostics.
- Firebase App Check where practical.
- Google Workspace login and company-domain restriction.
- Remote configuration of ring duration and related behavior.
- Improved retry and support tooling.

18.3 P2 - Future

- Current-holder tracking.
- Multiple target phones.
- Multiple offices.
- Department/location metadata.
- Request queue.
- ERP integration.
- Slack / Teams integration.
- Admin analytics and usage dashboard.
- Employee directory and automatic names.
- Target selection by user.

18.4 Explicit Non-Goals for MVP

- No Play Store publishing requirement.
- No Bluetooth hardware or pager system.

- No WhatsApp bot dependency.
- No third-party webhook platform.
- No full asset-management/current-holder workflow.
- No multi-device request queue.
- No requirement for every employee to install an app.

19. Implementation Sequence for Claude Code / Codex

19.1 Phase 0 - Bootstrap

- Create monorepo.
- Create Firebase dev project and environment structure.
- Create basic Android skeleton.
- Document environment variables and local setup.

19.2 Phase 1 - FCM Proof of Concept

Success criterion: An authenticated HTTP request containing "Rahul Sharma" results in the Samsung logging that Rahul Sharma requested the phone.

HTTP request -> Cloud Function -> Firebase Admin SDK -> FCM -> Android FirebaseMessagingService -> log requester name

Do not build polished web UI, acknowledgement, TTS, analytics, or advanced configuration before this path works.

19.3 Phase 2 - Physical Alert

FCM received -> deduplicate -> Delivered callback -> RingController -> notification + sound + vibration

Add a Test Ring button using the exact same RingController.

19.4 Phase 3 - Targetter Website

- First-time name entry.
- localStorage persistence.
- Firebase Auth.
- RING PHONE button.
- API integration.
- Basic error/loading states.

19.5 Phase 4 - Delivery Confirmation

Samsung receives FCM -> /delivered -> Firestore -> website realtime status

19.6 Phase 5 - Acknowledgement

Samsung holder presses I'VE GOT IT -> acknowledge API -> Firestore -> website realtime -> acknowledged

19.7 Phase 6 - Reliability

- Idempotency.
- Rate limiting.
- Atomic concurrency control.
- Expiration.
- Duplicate FCM protection.
- Timeouts.
- Token refresh.
- Network/reconnect handling.
- Structured error states.

19.8 Phase 7 - Production APK

- Create production Firebase configuration.
- Create versionCode/versionName policy.
- Sign release APK.
- Enable Crashlytics as appropriate.
- Document install/update steps.
- Perform locked-screen and background-delivery test matrix on the actual Samsung.

20. Acceptance Test Plan

Test	Setup / Action	Expected Result
A	First visit; enter Rahul Sharma	Name saved and main request screen shown
B	Close and reopen browser	Rahul shown without re-entering name
C	Samsung screen off; press Ring	Samsung receives request, sound/vibration fire, notification shows Rahul
D	Samsung locked	Visible notification and configured alert behavior still work
E	Holder taps I'VE GOT IT	Ring stops; request becomes acknowledged; browser updates without refresh
F	Double-click Ring	One request, one FCM send, one ring
G	Second employee requests during active request	Backend refuses second active request cleanly
H	No acknowledgement	Request expires; requester can try again
I	FCM token refreshes	New token registers automatically; future requests still work
J	Samsung offline	Website never claims Delivered; request waits/expires appropriately
K	Cancel active request	Request cancelled; matching target alert stops
L	Duplicate FCM payload	Android deduplicates and does not ring twice
M	Notifications disabled	Diagnostics clearly report issue
N	Test Ring pressed	Same RingController produces local alert without Firebase

20.1 Production Readiness Gate

- All PO acceptance tests pass on the actual shared Samsung.
- Release APK is signed and install/update process is documented.
- FCM token can refresh without manual database edits.
- Website does not expose privileged credentials or device token.
- Backend state transitions and idempotency have automated tests.
- The team can distinguish local audio problems from FCM/network problems using Test Ring and diagnostics.

21. Deployment and APK Distribution

21.1 Website

Deploy using the chosen Wordle/web-builder setup if it supports the required Firebase/HTTPS capabilities. Otherwise deploy the utility page using Firebase Hosting or another suitable host and attach the company subdomain.

21.2 Cloud Functions / Firestore

- Deploy dev first, run end-to-end tests, then promote configuration to prod.
- Use separate Firebase configuration/environment files where applicable.
- Do not share production service-account credentials with frontend code or coding agents in commits.

21.3 APK

For one company-owned target phone, Play Store distribution is unnecessary for MVP. Produce a release-signed app-release.apk and install it on the Samsung using ADB, USB, or a secure internal transfer method.

- Do not use a debug APK as the permanent installation.
- Keep release keystore and passwords secure and outside Git.
- Document how to update the APK without losing app configuration.
- After install: allow notifications, set notification volume appropriately, allow background data, and configure Samsung battery settings to avoid aggressive sleeping where possible.

21.4 Operational Configuration

Recommended production defaults: 10-second ring duration, 60-second request expiration, 20-second Targetter cooldown, single active request, vibration enabled, and TTS disabled until the core alert flow is proven stable.

22. Requirements Claude Code / Codex Must Not Violate

- Do not use Firestore polling as the primary target-device trigger.
- Do not put Firebase Admin credentials or server-side FCM credentials in the browser.
- Do not hardcode the target FCM token.
- Do not treat FCM send acceptance as handset delivery.
- Do not allow infinite ringing.
- Do not allow unlimited Ring spam.
- Do not allow clients to arbitrarily write delivered/acknowledged states.
- Do not build all product surfaces before proving FCM end to end.
- Do not make the Android app a WebView wrapper.
- Do not require every employee to install an app.
- Do not introduce a third-party webhook provider without a specific integration need.
- Do not create a request queue for MVP.

Guiding principle: First make one button reliably cause one Samsung to ring and show one person's name. Everything else comes after that.

Appendix A - API Contract Summary

Operation	Caller	Server responsibility	Result
Create request	Targetter web	Auth, validate, rate-limit, idempotency, Firestore create, FCM send	requestId + sent state
Delivered	Target APK	Validate target + request; transition sent -> delivered	deliveredAt
Acknowledge	Target APK	Validate target + state; transition delivered -> acknowledged	acknowledgedAt
Cancel	Targetter web	Validate ownership/state; transition to cancelled; optional cancel FCM	cancelledAt
Register device	Target APK	Validate/provision target and update latest FCM token	device updated

Operation	Caller	Server responsibility	Result
Heartbeat	Target APK	Update health timestamp safely	lastSeenAt

Appendix B - Core Shared Types

```

type PhoneRequestStatus =
  | "created"
  | "sent"
  | "delivered"
  | "acknowledged"
  | "cancelled"
  | "expired"
  | "failed";

interface PhoneRequest {
  requestId: string;
  clientRequestId: string;
  targetDeviceId: string;
  requestedBy: { uid: string; name: string };
  status: PhoneRequestStatus;
  createdAt: unknown;
  sentAt?: unknown;
  deliveredAt?: unknown;
  acknowledgedAt?: unknown;
  cancelledAt?: unknown;
  expiresAt: unknown;
  failureCode?: string;
}

data class IncomingPhoneRequest(
  val requestId: String,
  val requestedByName: String,
  val targetDeviceId: String
)

```

Appendix C - Copy-Paste Prompts for Claude Code / Codex

C.1 Architecture Review Prompt

Read PRD.md completely before making implementation changes.

The product is an internal shared-phone request system consisting of:

1. A Targetter web interface.
2. Firebase Cloud Functions backend.
3. Cloud Firestore state.
4. Firebase Cloud Messaging.
5. One native Android Target APK.

The most important architectural rule is:

FCM triggers the Target Android phone. Firestore realtime synchronizes state. Do not use Firestore polling as the primary ringing mechanism.

The website must call a backend HTTPS API / Firebase Cloud Function. The browser must never send FCM directly or contain Firebase Admin credentials.

Do not implement the complete product yet. First produce:

- proposed monorepo structure
- system architecture
- Firebase architecture
- API contracts
- Firestore schema
- Android module architecture
- environment variables
- security considerations

- implementation sequence
- testing strategy
- major Android background-delivery risks

Identify assumptions or contradictions in the PRD. Do not write production code until the architecture plan has been reviewed.

C.2 First End-to-End Proof Prompt

Implement only the first end-to-end technical proof.

Goal:

HTTP request -> Firebase Cloud Function -> Firebase Cloud Messaging -> Android
 FirebaseMessagingService -> Samsung logs requester name.

Do not build the complete web UI. Do not implement acknowledgement yet.

Build:

- Firebase Functions TypeScript project
- basic target device registration
- FCM send service
- Android Kotlin project
- Firebase Messaging integration
- token registration
- incoming FCM handling
- diagnostic logging

Add setup instructions. Stop when an HTTP request containing "Rahul Sharma" can result in the Samsung logging that Rahul Sharma requested the phone.

C.3 Physical Alert Prompt

Extend the existing working pipeline. Do not rewrite the architecture.

Add:

- custom notification channel
- distinctive bundled ringtone
- vibration
- request ID deduplication
- notification displaying requester name
- Test Ring diagnostic button

Success criterion:

API -> FCM -> locked Samsung -> sound + vibration + requester name.

C.4 Targetter Website Prompt

Build the Targetter web experience.

Requirements:

- first-time name entry
- localStorage name persistence
- Firebase authentication
- Ring Phone button
- UUID idempotency key
- create request API
- loading/error states
- Firestore realtime request listener
- Sent state
- Delivered state
- Acknowledged state

Keep the UI extremely simple and responsive.

C.5 Acknowledgement Prompt

Add the complete acknowledgement pipeline:

Samsung receives FCM

-> calls Delivered API

-> browser displays Phone Received Request

Samsung user presses I'VE GOT IT
 -> calls Acknowledge API
 -> Firestore status changes
 -> browser displays Acknowledged

Implement state transition validation server-side. Add tests for duplicate acknowledgements and stale requests.

C.6 Backend-Only Phase Prompt

Implement the backend phase only.

Configure:

- Firestore schema
- Firestore security rules
- Firebase Functions TypeScript project
- Firebase Admin initialization
- shared TypeScript types
- createPhoneRequest
- validation
- idempotency
- request cooldown and single-active-request enforcement

Do not implement the Android application yet. Add unit tests for backend request creation and state transitions. Stop after this phase and provide setup instructions and tests.

Appendix D - Final Build Checklist

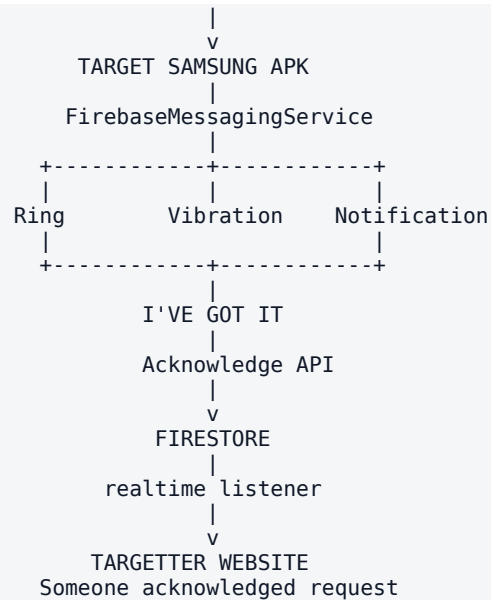
Area	Checklist
Web	Name persistence; auth; Ring Phone; UUID; loading/error states; realtime Sent/Delivered/Acknowledged; cancel; responsive UI
Backend	Cloud Functions; Admin SDK; validation; idempotency; rate limit; atomic concurrency; state machine; FCM service; logs
Firestore	users; devices; phoneRequests; config; requestEvents; rules; indexes as needed
FCM	High priority; current target token; minimal payload; send errors handled; cancel payload optional
Android	Kotlin; FirebaseMessagingService; token refresh; dedupe; notification; ring; vibration; acknowledge; diagnostics; Test Ring
Security	No Admin credentials in web; no FCM token exposure; server timestamps; controlled device registration; secrets outside Git
Reliability	Locked-screen tests; offline tests; duplicate tests; token-refresh tests; expiry; cancellation; single active request
Release	Production Firebase config; signed APK; install guide; Samsung notification/battery settings; rollback/update process

Appendix E - Final Architecture in One View

```

TARGETTERS (~50 EMPLOYEES)
Browser / phone.arnifi.com
  |
  | HTTPS request
  v
FIREBASE CLOUD FUNCTIONS v2
  |
  +--> Auth / validation / cooldown / idempotency / state machine
  |
  +--> FIRESTORE: source of truth
  |
  +--> FIREBASE ADMIN SDK -> FCM

```



Final source of truth: The implementation should follow the architecture and priority order in this document. Any deviation proposed by Claude Code or Codex should be explained before code is changed, especially if it affects FCM delivery, security, Android background behavior, or request state semantics.